BANKING MANAGEMENT SYSTEM

Software Design Description

Architecture, record formats, locking, protocol, and operation flows

Implementation-specific documentation

Important: Features and limitations are documented from the project source files and README. Tests are not claimed as independently executed for this documentation package.

1. Design overview

The system uses procedural C and POSIX operating-system interfaces rather than C++ classes, SQL tables, or a web framework. The command table maps an action name to permitted role, expected argument count, and handler function. The file layer implements fixed-width records and synchronization explicitly.

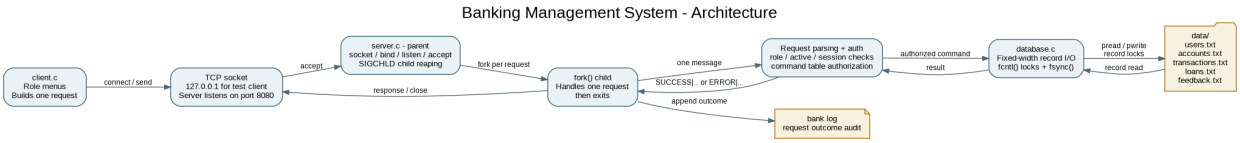


Figure 1. High-level runtime architecture.

2. Module responsibilities

Component	Responsibilities
client.c	Role selection, login prompts, role menus, command construction, TCP connection per command, response parsing/display.
server.c	Create/bind/listen/accept TCP socket; fork per request; parse request; authenticate and authorize; command table; business operations; audit logging; SIGCHLD handler.
database.c	Table descriptors; record structures; file open/close; fixed-width read/write; record lookup; record and table locks; seed initialization and stale-session reset.
stress_test.c	Creates concurrent test processes and checks login exclusivity, concurrent deposits, bounded withdrawals, and transfer invariants.
data/*.txt	Authoritative persisted user/account/transaction/loan/feedback records.
bank.log	Timestamped request outcomes appended by the server.

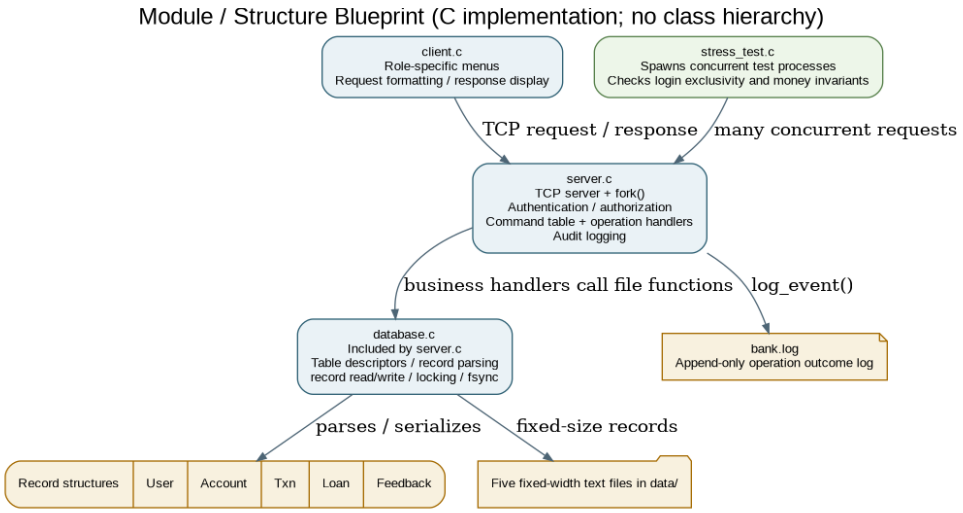


Figure 2. Module and structure blueprint.

3. Process lifecycle

- The parent calls db_init(), creates an IPv4 TCP socket, sets SO_REUSEADDR, binds INADDR_ANY:8080, and listens with backlog 10.
- For every accepted connection, the parent calls fork().
- The child closes its copy of the listening socket, opens its own data-file descriptors, reads one request, runs validation and a handler, writes a response, closes the connection, logs the event, and exits.
- The parent closes its copy of the connected socket and continues accepting.
- The parent installs a SIGCHLD handler that loops over waitpid(-1, NULL, WNOHANG), reaping terminated children.

4. Network protocol

Requests are pipe-delimited text. Login and customer registration carry a username/password instead of the authenticated user ID. Most other actions include action, client-selected role, user ID, and zero or more arguments. The server then verifies user state and dispatches through the command table.

```
LOGIN|2|emp1|pass
TRANSFER|1|101|10002|50
LOGOUT|1|101
```

Replies use SUCCESS|... for successful operations and ERROR|... for rejected/failed operations. The server reads one request using a single recv() and returns one response, then closes the connection. This is the intended message model described by the implementation; TCP may split/coalesce data, so a robust future protocol should use a length prefix or delimiter framing with a read loop.

5. Authorization and command routing

The server looks up the user by stored ID. For authenticated commands it verifies that the session is active, the user is active, and the user's stored role matches the role supplied in the request. A command table associates action names with role IDs, argument counts, and handler functions. Role 0 is used for commands such as CHANGE_PASS that all authenticated roles may use. Loan processing additionally verifies the assigned employee ID and that loan status is ASSIGNED.

Role	ID	Command families
Customer	1	VIEW_BALANCE, DEPOSIT, WITHDRAW, TRANSFER, VIEW_HISTORY, APPLY_LOAN, MY_LOANS, ADD_FEEDBACK, CHANGE_PASS
Employee	2	LIST_CUSTOMERS, ADD_CUSTOMER, MOD_CUSTOMER, VIEW_PASSBOOK, VIEW_ASSIGNED_LOANS, PROCESS_LOAN, CHANGE_PASS
Manager	3	LIST_CUSTOMERS, ACT_DEACT_ACC, VIEW_LOANS, LIST_EMPLOYEES, ASSIGN_LOAN, REVIEW_FEEDBACK, CHANGE_PASS
Administrator	4	LIST_USERS, ADD_EMPLOYEE, MOD_USER_DETAILS, MANAGE_ROLES, CHANGE_PASS

6. File schema and serialization

Record type / file	Record bytes	Logical fields
User / users.txt	128	id, name, pass, role, active, logged_in
Account / accounts.txt	128	account id, user id, balance
Transaction / transactions.txt	160	transaction id, account id, type, amount, details, timestamp
Loan / loans.txt	128	loan id, account id, amount, status, employee id
Feedback / feedback.txt	384	feedback id, user id, timestamp, message

Records are delimited by "|" and padded with spaces to their file's fixed record size. Reads use pread() at record_index × record_size. Writes use pwrite() and then fsync(). Appends lock the whole file (a byte-range lock of length zero means to the end of the file).

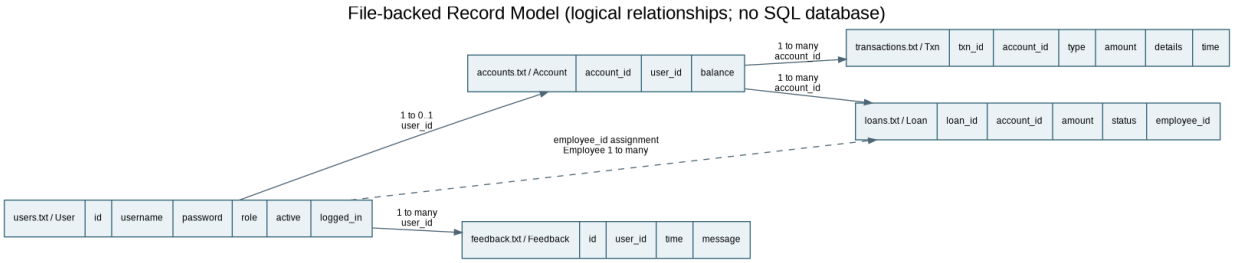


Figure 3. Logical record relationships; there are no database-enforced foreign keys.

7. Synchronization design

Operation	Lock strategy	Reason
Update one user/account/loan	Exclusive byte-range record lock with F_SETLKW	Prevent concurrent writers from changing the same record simultaneously.
Add records	Whole-file write lock; count records; append; unlock	Avoid duplicate record IDs and conflicting append positions.
Transfer	Lock lower account record index first, then higher; re-read both; update; unlock reverse order	Avoid deadlock between opposite-direction transfers and avoid stale balance decisions.
Session login	Lock user record around check/update of logged_in	Ensure competing logins cannot both claim the same session.
Write durability	pwrite() followed by fsync()	Request persistence of each record write before treating it as complete.

These are application-managed guarantees. Unlike a transactional database, the text-file design does not provide a write-ahead log or crash-recovery transaction covering a multi-file or multi-record operation.

8. Transfer design

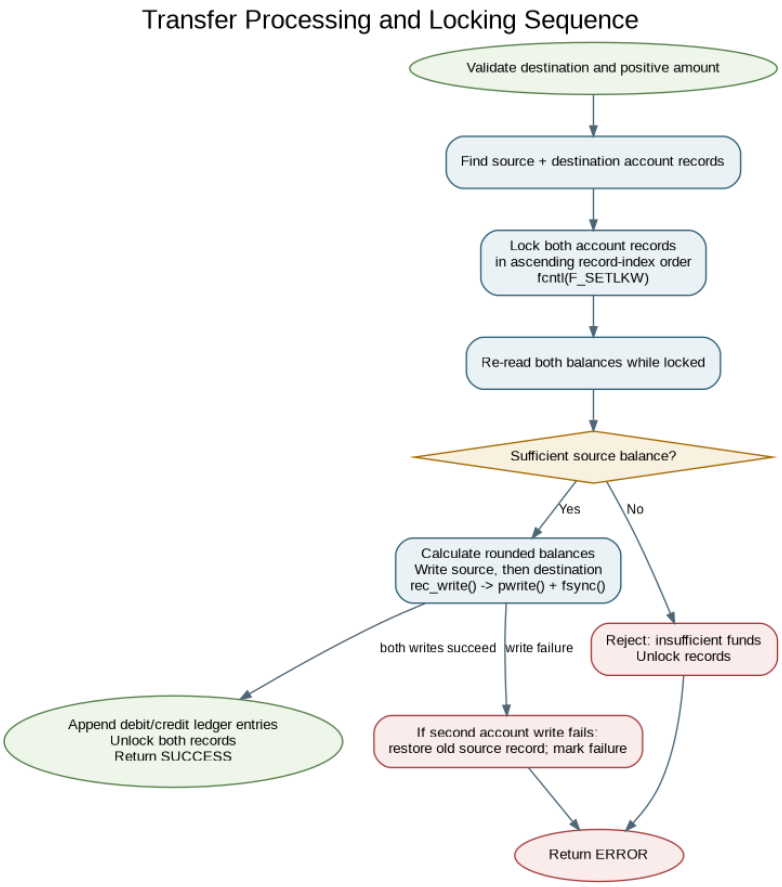


Figure 4. Transfer validation, locking, and compensating rollback.

A transfer locates both account records, rejects invalid/self-transfer/inactive target, obtains both record locks in a deterministic order, re-reads the data, checks available funds, computes rounded balances, writes source then destination, and attempts to restore the old sender record if the destination write fails. It then records transfer-related ledger entries and releases locks.

Important limitation: restoring the sender record handles an observed destination-write error, but does not make the pair of writes crash-atomic. A process crash between the two writes can leave a mismatch. “Durability” at individual record level is not identical to transaction atomicity.

9. Loan lifecycle

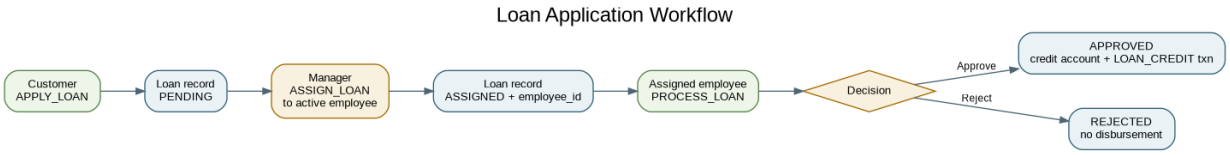


Figure 5. Loan application, assignment, and decision path.

A customer applies and receives PENDING status. A manager assigns the loan to an active employee, changing it to ASSIGNED and storing employee_id. The assigned employee may mark it APPROVED or REJECTED once. On approval, the loan amount is credited to the account and a LOAN_CREDIT transaction is recorded; if the credit attempt fails, status is restored to ASSIGNED.

10. Operational limits and security posture

- Plaintext password storage and direct password comparisons are present in the current implementation.
- The TCP channel is not encrypted or authenticated at the transport layer.
- The server binds to INADDR_ANY, so host firewall/network configuration matters.
- The text protocol uses delimiter-separated arguments with no general escape mechanism.
- Single recv() request parsing is not robust to arbitrary TCP packetization.
- Amounts use double, which can introduce rounding representation concerns for money.
- The storage layer uses linear record scans, suitable for project-sized files but not large volumes.
- A partial open failure in db_open() and general error-path handling should be reviewed for resource cleanup.
- Customer creation writes the user record and account record under file locks but is not backed by a durable multi-record transaction; a failure on the second write can leave partial creation.